

From LLMs to Multi-Agent Systems

A Progressive Guide to Agent Architectures

J.C. Vaught

University of South Carolina

February 3, 2026

Outline

Foundation

Agent Workflows

Slaves and Masters

Communication Patterns

Control Models

Orchestration Cycles

Topologies

Canonical Patterns

Multi-Layered Architectures

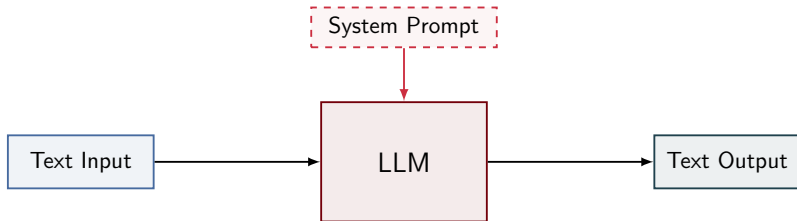
Pattern Selection

Conclusion

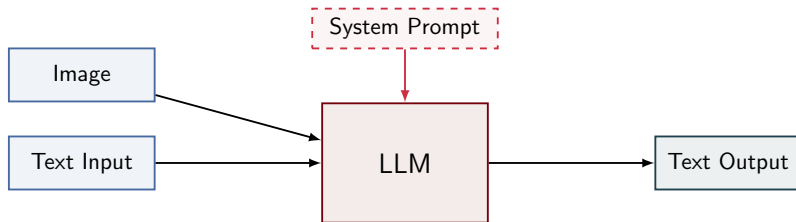
What Is an LLM?

- ▶ Text in, text out — a prediction engine
- ▶ No persistent memory between conversations
- ▶ System prompt shapes behavior (invisible to user)
- ▶ Size-capability tradeoff: small/fast vs. large/capable

LLM: Basic Interface



Multimodal LLM



Can see images, but output is still text only

AI-Assisted Coding (Autocomplete)

- ▶ LLM predicts code continuation
- ▶ No access to other files, tools, or execution
- ▶ User accepts or ignores suggestions

Before

```
def main():  
    print("Hel|
```

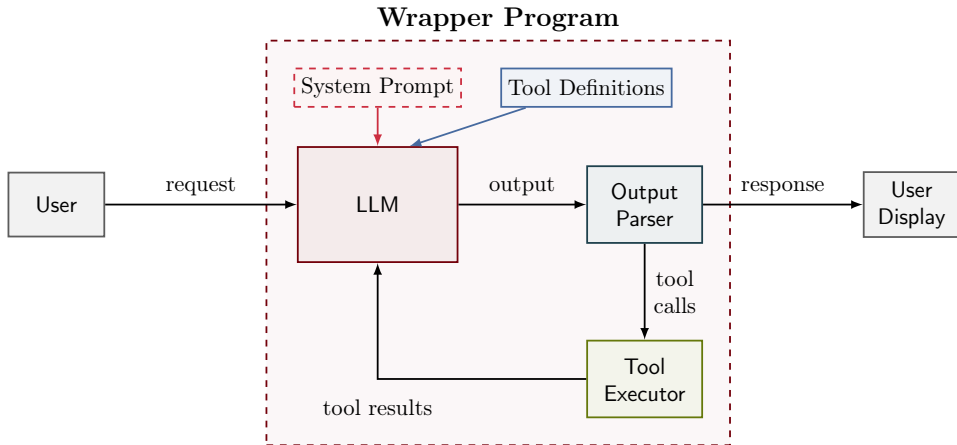
After

```
def main():  
    print("Hello, World!")|
```

What Is an AI Coding Agent?

- ▶ LLM + wrapper program = agent
- ▶ Wrapper provides: tool definitions + execution environment
- ▶ LLM outputs tool calls → wrapper executes → results fed back
- ▶ Creates an action loop until task complete

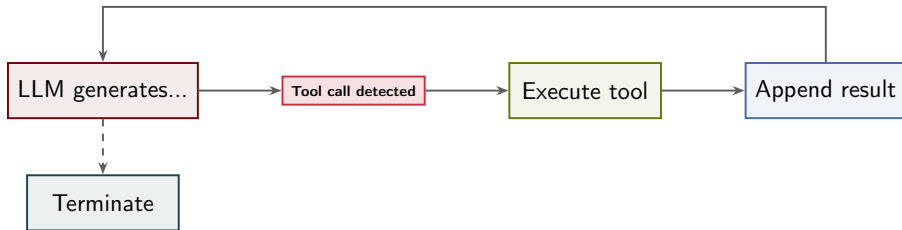
Agent Architecture



Three Execution Models

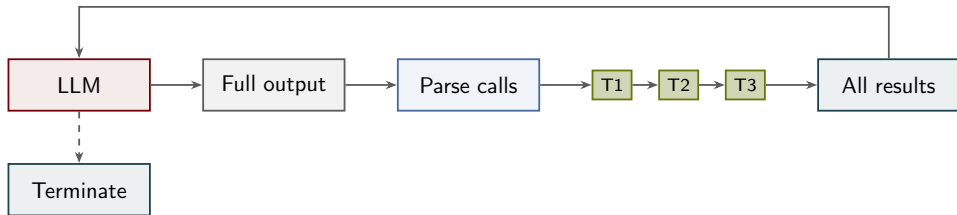
- ▶ **Streaming:** Execute tools as soon as detected mid-generation
- ▶ **Batch:** Generate full response, then execute all tools
- ▶ **Parallel:** Some tools run concurrently, others block

Streaming Execution



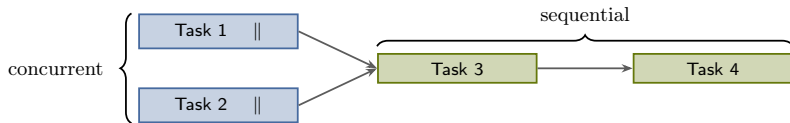
Real-time, responsive — tools fire immediately

Batch Execution



Simpler to implement — classic chatbot model

Parallel Execution

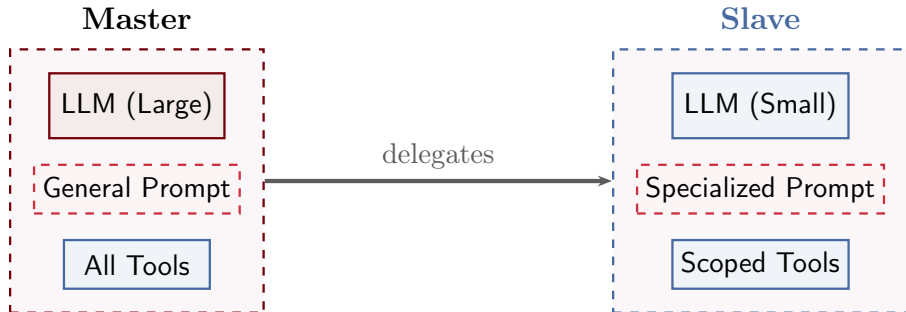


Independent tasks overlap; blocking tasks wait

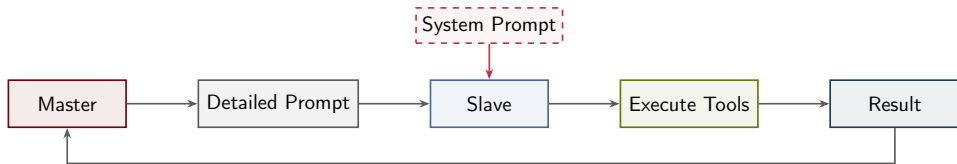
What Is a Slave?

- ▶ Same architecture as an agent
- ▶ Called by another agent (master), not by a human
- ▶ Often uses smaller/specialized model
- ▶ Scoped tools and specialized system prompt

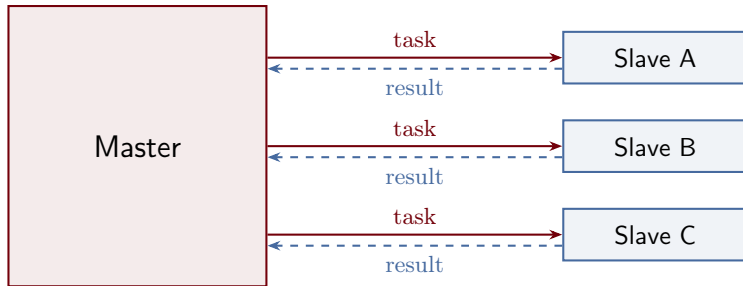
Master vs. Slave



Slave Lifecycle



Master-Slave Pattern



Communication: Direct Return



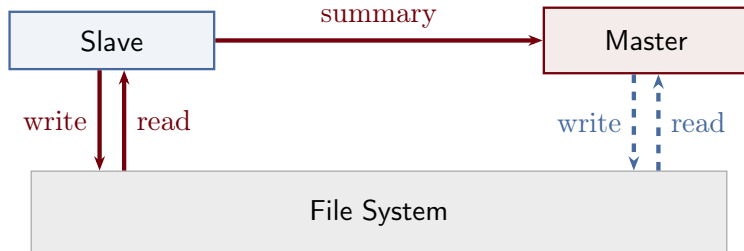
Result embedded in completion message
Best for small results (pass/fail, status codes)

Communication: File-Mediated



Best for large artifacts, binaries, generated files

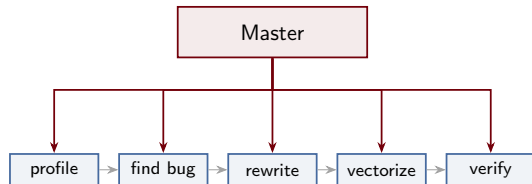
Communication: Hybrid (Most Common)



Detailed artifacts to disk + summary to master

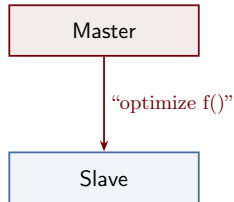
Imperative vs. Declarative Control

Imperative



many small, cheap workers
one task each

Declarative



one capable, expensive slave
autonomous reasoning

Control Model Comparison

Property	Imperative	Declarative	Hybrid
Intelligence location	Master	Slave	Shared
Slave autonomy	Low	High	Medium
Predictability	High	Low	Medium
Flexibility	Low	High	Medium

The Master Cycle

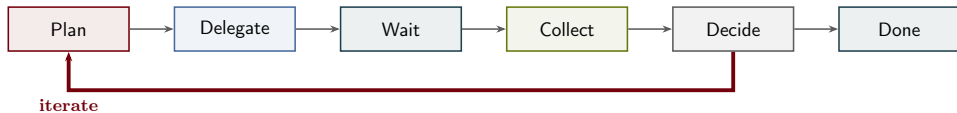
- ▶ Five-step loop: **Plan** → **Delegate** → **Wait** → **Collect** → **Decide**
- ▶ Differences lie in how each step executes
- ▶ May traverse once (single-pass) or repeat (iterative)

Single-Pass Orchestration



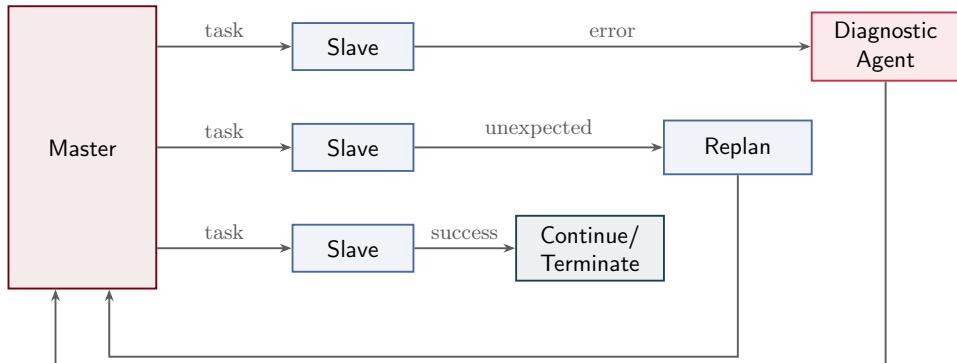
One traversal, no recovery, no refinement
If initial plan is wrong, no recourse

Iterative Orchestration



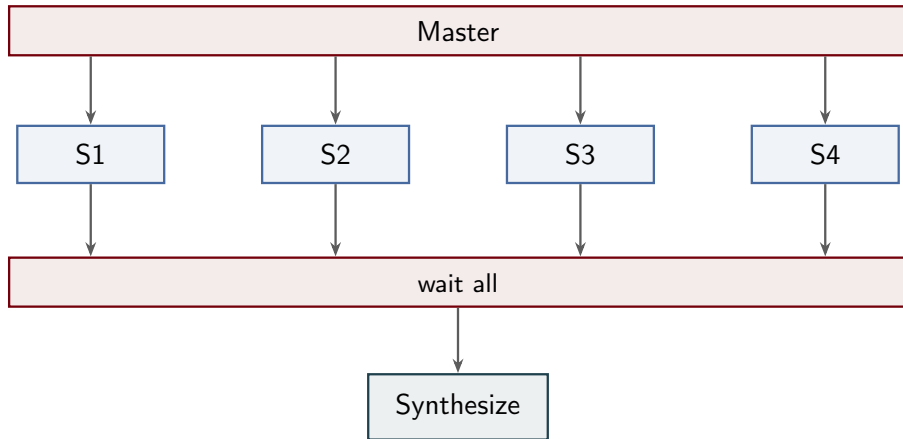
Feedback loop enables error recovery and refinement

Reactive Orchestration



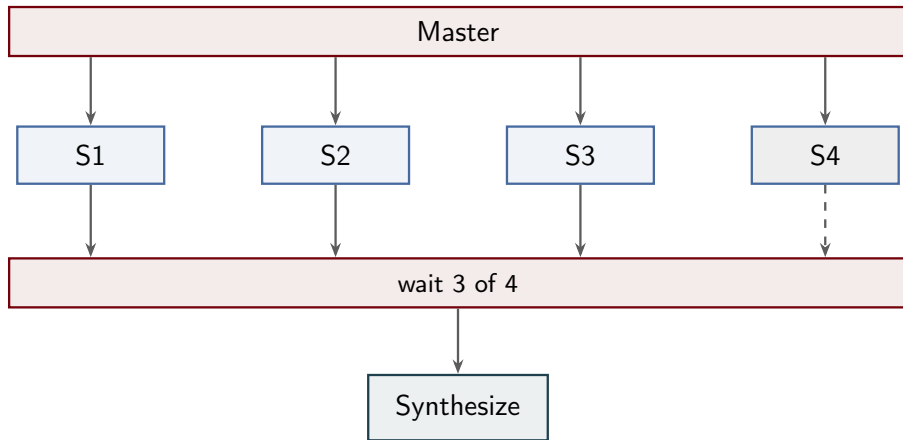
Event-driven: react to success, error, or unexpected discoveries

Parallel Agent Execution: Full Barrier



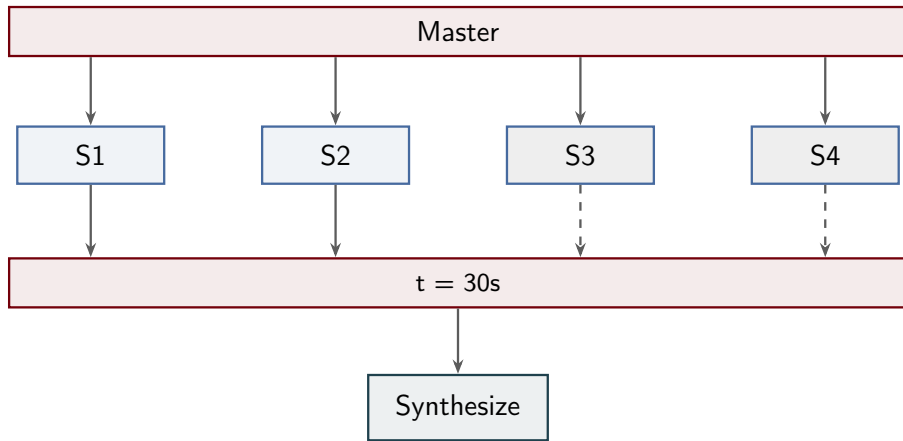
Slowest slave determines total time

Parallel Agent Execution: Partial Barrier (k-of-n)



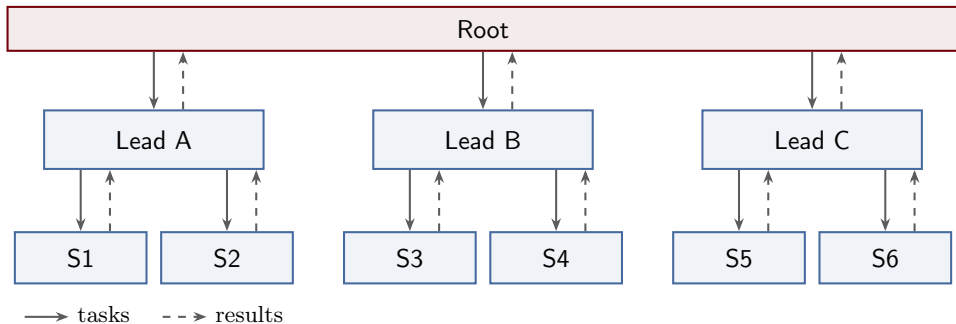
Proceed when k slaves finish; grayed slave not required

Parallel Agent Execution: Timeout Barrier



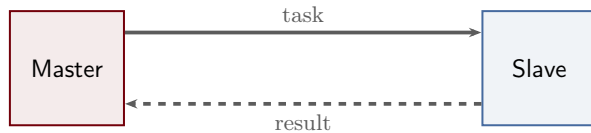
Prioritizes responsiveness over completeness

Tree-Structured Hierarchy

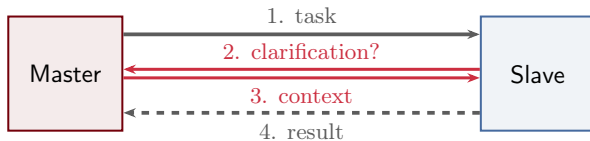


Unidirectional vs. Bidirectional Communication

Unidirectional:



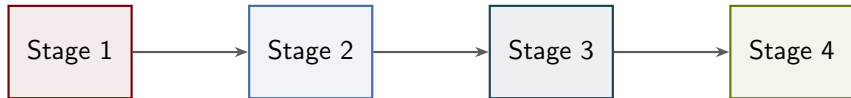
Bidirectional:



Four Canonical Patterns

- ▶ **Pipeline:** Serial, single-pass, unidirectional
- ▶ **Fan-Out/Fan-In:** Parallel, single-pass, unidirectional
- ▶ **Master-Slave Pool:** Parallel, iterative, unidirectional
- ▶ **Feedback Loop:** Serial, iterative, bidirectional

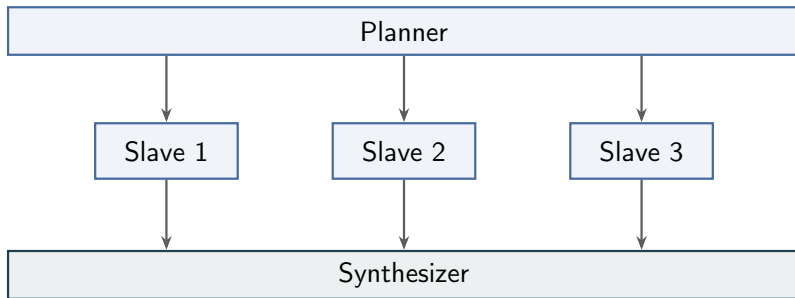
Pipeline Pattern



Sequential transformation chain

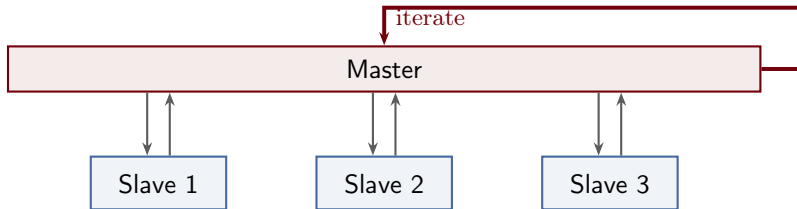
Latency = sum of all stages

Fan-Out/Fan-In Pattern



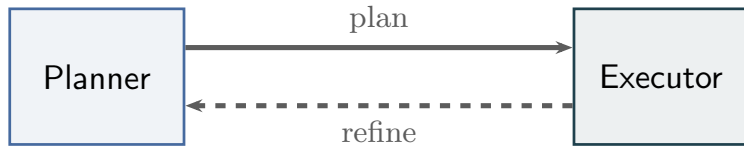
Independent subtasks in parallel
Latency = max of workers

Master-Slave Pool Pattern



Task queue + iterative processing
Best for batch workloads

Feedback Loop Pattern

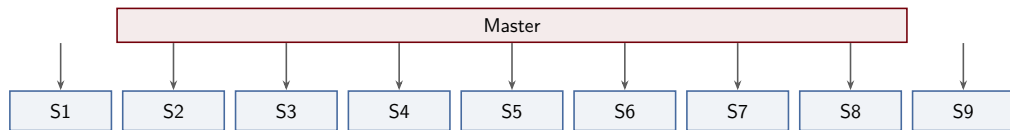


Iterative refinement until quality threshold
Best for tasks requiring multiple attempts

Why Multi-Layered?

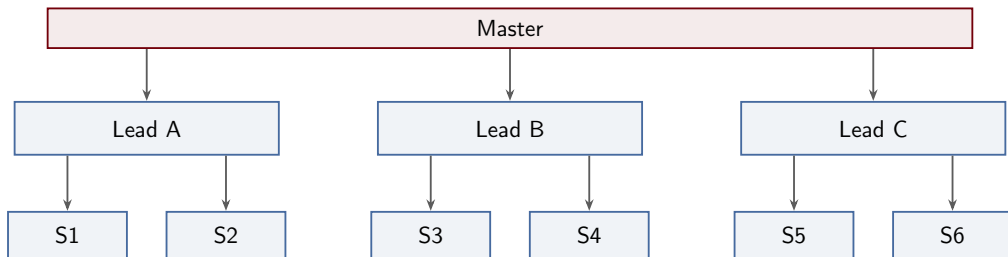
- ▶ Flat master = bottleneck at scale
- ▶ Context fills with operational details
- ▶ Layering separates: Strategy $\rightarrow$ Planning $\rightarrow$ Execution
- ▶ Each layer filters and abstracts information

Flat Architecture (Two-Layer)



Master holds both strategic and tactical context

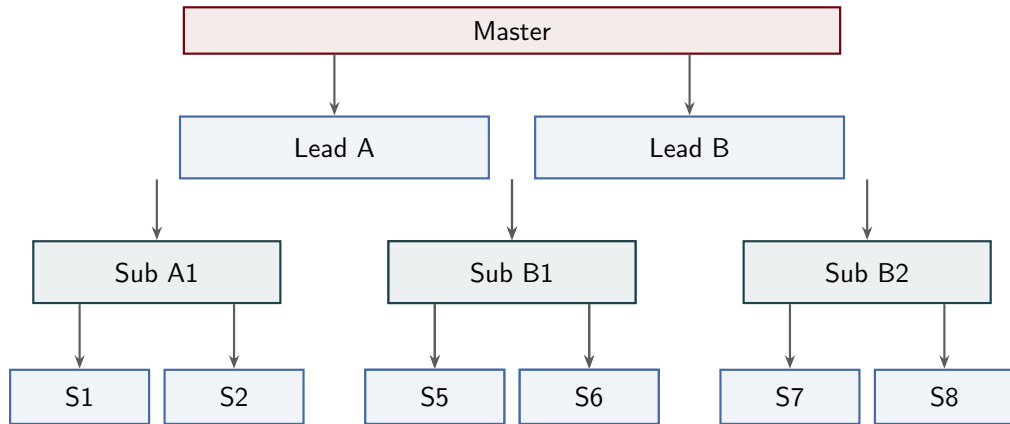
Layered Architecture (Three-Layer)



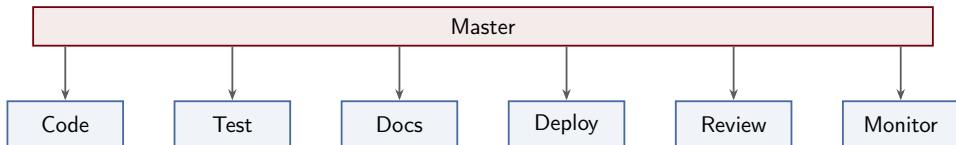
No single agent holds entire system state

Deep vs. Wide Architecture

Deep (4 layers)



Wide Architecture

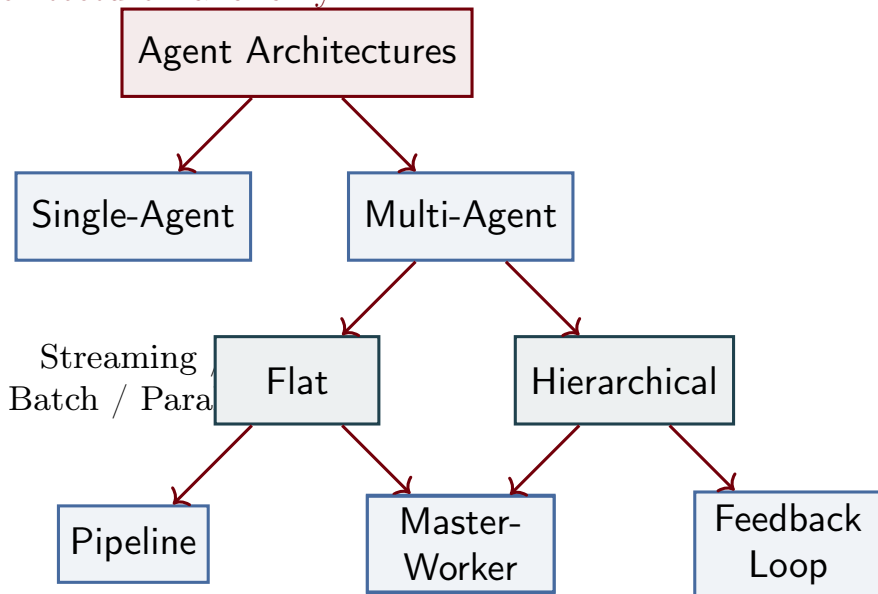


Fewer handoffs mean lower latency
Best for domain coverage requiring specialized expertise

Pattern Selection Guide

Criterion	Pipeline	Fan-Out	Master-Slave	Feedback
Independence	Low	High	High	Low
Iteration	No	No	Optional	Yes
Scale	3–5	3–20	10–100+	2–5
Latency	Sum	Max	Batched	Iterations
Best for	Transforms	Parallel work	Batch	Refinement

Agent Architecture Taxonomy



Key Takeaways

- ▶ LLM is an uncommitted tool — power comes from composition
- ▶ Four canonical patterns: Pipeline, Fan-Out, Master-Slave, Feedback Loop
- ▶ Architecture emerges from task structure, not model capability
- ▶ Patterns are composable — real systems combine them
- ▶ Multi-layered hierarchies scale beyond single-agent limits

Questions?

J.C. Vaught
jvaught@sc.edu
University of South Carolina